

Project Report: LiveCollab Data Visualization Platform

Prepared from repository review of the frontend, backend, data pipeline, and collaboration features.

1. Executive Summary

This project is a collaborative data analytics and dashboarding application designed to let users upload datasets, transform them into visual charts, and work together in shared workspaces. The repository has two major layers: a React + Vite frontend in the *live collab* folder and an Express + MongoDB backend in the root *server* folder. The application centers on user authentication, workspace management, dataset ingestion, chart building, dashboard creation, real-time chat, and version history. The architecture is ambitious and complete in many areas, but the codebase also shows several integration gaps, naming inconsistencies, and security considerations that must be addressed before production rollout.

At a high level, the platform allows teams to create a workspace, upload CSV/JSON/XLS/XLSX files, inspect dataset columns, build charts using rows and columns, and arrange dashboard cards. The frontend states are managed with Redux Toolkit, while the backend exposes routes for authentication, workspaces, datasets, dashboards, and charts, using MongoDB for persistence. Real-time collaboration is handled through Socket.IO, which supports chat and live dashboard events. The project therefore combines traditional business intelligence concepts with collaborative workspace features often seen in modern SaaS dashboards.

2. Product Scope and Functional Vision

The project is meant to function as a shared analytics environment rather than a static chart page. Users can sign up or sign in, create workspaces, upload data, browse available columns, drag fields onto chart wells, and generate interactive visualizations. There is also a dashboard canvas for arranging charts, multi-tab dashboard management, toolbar operations, share modal, version tracking, and chat. This indicates a broad product scope that includes both data exploration and collaborative decision-making.

The frontend folder structure reveals the expected modules for a mature application: pages, routes, layouts, components, hooks, services, UI state slices, and utilities. The project includes authorization screens, dashboard pages, upload flows, chart cards, collaboration drawers, and modal-based workflows. The design also reflects a modern dashboard UI with panels, filters, tools, and reusable visual primitives. This is a clear demonstration of an application designed to support both individual analytics tasks and team-based analysis in a single product surface.

Key product themes include:

- Authentication and protected routing for workspaces.
- Data ingestion from file uploads with parsing and validation.
- Chart generation based on field selection, aggregation, and sorting.
- Shared dashboard composition and layout controls.
- Collaboration via chat and member awareness.
- Version history and revision snapshots for dashboards.

3. Frontend Architecture and UI Composition

The frontend is built with React 19, Vite, Redux Toolkit, React Router, Socket.IO client, and Recharts. The application entry point is *src/main.jsx*, which renders the main app inside React StrictMode. The root application component in *src/App.jsx* wraps the UI in an ErrorBoundary, Redux Provider, SocketProvider, BrowserRouter, and Toaster notification handler. This structure ensures a global error boundary, centralized state, live connection support, and routing across the dashboard experience.

Routing is defined in *src/routes/AppRoutes.jsx*. It implements lazy loading for sign-in, sign-up, workspace page, and 404 page components. The app redirects old login routes to the sign-in path and protects workspace routes behind a PrivateRoute component. This pattern is standard for secure single-page applications, where a user must be authenticated before accessing workspace data. The page architecture is shaped around a main workspace view, including a sidebar, header, toolbar, data panel, canvas, and collaboration drawers.

The UI is divided into feature areas. Authentication components live in *src/components/Auth*, while core workspace features are organized under *src/components/Charts*, *DataPanel*, *DashboardCanvas*, *Members*, *Chat*, *VersionHistory*, *Toolbar*, *Header*, and *WorkspaceSidebar*. This modularization is strong because it supports clear separation of responsibilities and better future maintenance. The workspace page itself appears central to the product, acting as the control center where the user interacts with datasets, chart logic, collaborative tools, and dashboard state.

The project also uses a collection of custom hooks: *useAuth*, *useDebounce*, *useDragDropField*, *useOutsideClick*, *useSocket*, and *useZoomPan*. These hooks support common behaviors such as authentication state access, drag-and-drop field interactions, chart scaling, socket subscriptions, and outside-click detection. In parallel, Redux slices handle auth, workspace, dashboard, chart, chat, version, upload, and UI states. The store is configured with *serializableCheck* disabled, which is a common tradeoff in real-time apps but means state validation is handled more loosely than in stricter environments.

4. Backend, Data Layer, and API Design

The backend is an Express application running under the root *server* folder. The entry file, *server.js*, loads environment variables, connects to MongoDB, initializes Passport, mounts API routes, configures CORS, and starts the application with Socket.IO. This is the server spine of the system and establishes the application-level service boundaries. The database connection is handled by *server/config/mongo.config.js*, while authentication and Google OAuth configuration are managed in *passportconfig.js*.

The backend organizes logic around route modules and controllers: auth routes, workspace routes, dataset upload routes, dashboard routes, and chart routes. Models exist for users, workspaces, datasets, dashboards, charts, and messages. The dataset model stores file metadata, Cloudinary URL, schema details, and parsed row data. Dashboard and chart models support analytics objects, chart references, and version snapshots. This design is appropriate for a collaborative BI platform because it reflects both user ownership and data resource tracking.

The project also includes Cloudinary integration with Multer for temporary file handling and remote asset storage. Uploaded files are accepted only if they are CSV, JSON, XLS, or XLSX and under 5 MB. This ensures a manageable upload pipeline and reduces file-size-related issues. After upload, files are parsed and saved to MongoDB with metadata, which allows the frontend to create charting inputs from structured columns. In this sense, data import is not just storage but a transformation step into a chartable dataset model.

The backend is broad and feature-rich, but it also exposes notable inconsistencies. Some route-level protections are missing or incomplete, and there are naming mismatches between schema fields and controller logic. The user IDs are sometimes read directly from URL parameters rather than from decoded JWT claims. Several field names also differ between frontend and backend models, including dashboard versionHistory references, chart schema keys, and socket event naming. These issues are not cosmetic; they can break feature flows and create runtime failures if not corrected.

5. Data Flow: Upload, Charting, and Collaboration

The core data lifecycle begins with a user selecting a workspace and uploading a dataset. The client calls a service method that sends a multipart upload request with progress tracking. On the server, Multer stores the file temporarily and then the dataset controller parses it with compatible libraries such as PapaParse or XLSX. This parse stage creates column definitions and rows, which are saved to MongoDB for later use in charts and analytics. The file also gets pushed to Cloudinary for remote storage and retrieval.

Once data is loaded, the frontend allows the user to drag or select fields from the workspace tree into axis wells such as rows, columns, filters, or metrics. A chart slice stores this builder state, and the application transforms that selected structure into chart data. The charting layer uses Recharts, enabling bar, line, pie, donut, area, scatter, and other chart types. The system is designed to aggregate data by operations such as sum, average, count, min, and max. This is a classic analytics workflow where the user chooses dimensions and measures and then builds a dashboard object from the resulting visual structure.

Real-time collaboration is implemented through Socket.IO. The socket provider establishes a connection when a token exists, and the chat drawer joins specific workspace rooms to send and receive messages. The server configuration includes sockets for chat, dashboard updates, and chart events. This design is well aligned with the platform's collaborative goal, because multiple users can work in the same workspace and see messages, chart changes, or version updates in real time. In addition, the project contains version history logic for dashboards and chart snapshots, which supports review, comparisons, and restoration of previous states.

Although the collaboration model is intentionally rich, there are a few implementation problems worth noting. The current frontend may duplicate messages by inserting a local message and then receiving a server-broadcasted message with the same payload. Some event names and socket handlers are not fully aligned. Also, the integration notes indicate that some collaboration features, such as typing indicators, member presence, and version restore actions, are partially implemented or not yet fully consumed by the UI. These are important concrete areas for stabilization before the project reaches production quality.

6. Risks, Gaps, and Production Readiness

The project is structurally impressive and clearly demonstrates substantial engineering effort. The component decomposition, Redux organization, API route setup, and collaborative architecture all point to a serious product vision. However, it is not yet fully production-ready. The integration notes explicitly call out mismatched API URLs, unclear service constants, case-sensitive file path issues, inconsistent endpoint patterns, and auth behavior problems. Examples include using a frontend base URL of localhost:8000 while the backend listens on port 5000, missing or misnamed constants, route ordering problems, and a commented-out socket authentication middleware.

There are also security and data integrity concerns. Route handlers do not always enforce workspace ownership or role-based restrictions, chart and dataset operations can be performed without full verification, and Google OAuth tokens are stored in URLs in a way that could leak through logs and browser history. In stricter systems, all sensitive operations should derive the user context from verified JWT claims and should enforce membership checks before returning or modifying data.

From a delivery perspective, the codebase needs stronger testing and validation. There is no visible automated test suite, route validation layer, or end-to-end regression checks. The backend and frontend also need better contract alignment: chart shapes, response data structures, route path conventions, and event payloads must be standardized. If these gaps are fixed, the system could become a strong collaborative analytics platform with strong visual tooling and team-based reporting capabilities.

Overall, the project demonstrates a solid concept and an impressive technical foundation. It successfully combines visual analytics, shared workspaces, real-time collaboration, and data management in one application. With focused cleanup around auth consistency, route security, naming conventions, and UI/backend data contract alignment, the project has a credible path toward a production-grade data collaboration platform.

Conclusion

This repository represents a feature-rich analytics collaboration application with realistic enterprise ambitions. Its architecture is broad, modular, and largely coherent, spanning frontend state, backend service logic, file ingestion, real-time messaging, and dashboard editing. The most important next step is not adding more features, but stabilizing the existing integrations so the project becomes reliable, secure, and consistent across layers.